PropOS Phase 1 专项测试计划

> 版本: v1.2
>
> 日期: 2026-04-13
>
> 对应文档: PRD v1.8 / ARCH v1.5 / data*model v1.5* / *API*CONTRACT v1.8 / IMPLEMENTATION_CHECKLIST v1.7
>
> 范围: Phase 1 全模块（M1~M5 + KPI + 基础底座）

目录

一、测试目标与原则

- 1.1 目标
- 1.2 原则

二、测试分层策略

三、覆盖率目标

四、单元测试计划

- 4.1 租金递增引擎 (`packages/rent_escalation_engine`)
- 4.2 KPI 打分引擎 (`packages/kpi_scorer`)
- 4.3 WALE 计算 (`wale_service.dart`)
- 4.4 NOI 计算 (`noi_service.dart`)
- 4.5 信用评级 (`credit_rating_service.dart`)
- 4.6 核心 Service 层测试

五、集成测试计划

- 5.1 数据库迁移验证
- 5.2 Repository 集成测试
- 5.3 事务原子性测试

六、API 端到端测试计划

- 6.1 认证链路
- 6.2 RBAC 权限验证
- 6.3 响应信封验证
- 6.4 核心业务链路 E2E

七、状态机转换测试矩阵

- 7.1 合同状态机（7 态 12 条转移）
- 7.2 工单状态机（7 态 11 条转移）
- 7.3 账单状态机（6 态 11 条转移）
- 7.4 押金状态机（4 态 7 条转移）

八、业务计算精度测试
8.1 WALE 验收样本
8.2 NOI 验收样本
8.3 递增规则验收样本
8.4 KPI 打分验收样本
九、安全测试计划
9.1 OWASP Top 10 清单
9.2 二房东数据隔离专项
9.3 文件安全
9.4 PIPL 合规验证
十、性能测试计划
10.1 API 响应时间 SLA
10.2 批量操作性能
10.3 并发测试
10.4 数据库查询优化验证
十一、前端测试计划（Flutter + Admin）
11.1 BLoC/Cubit 单元测试（Flutter）+ Pinia Store 单元测试（Admin）
11.2 组件测试
11.3 状态色块映射验证
十二、测试数据策略
12.1 种子数据覆盖范围
12.2 边界值测试数据
12.3 Mock 数据生成规则
十三、UAT 验收标准
13.1 资产模块
13.2 租务与合同
13.3 财务与 NOI
13.4 工单
13.5 二房东穿透
13.6 KPI 考核
13.7 安全验收
十四、定时任务测试
14.1 任务执行正确性
14.2 幂等性与防重
14.3 失败与重试
14.4 执行顺序依赖

十五、回归测试清单
十六、工具链与运行方式
16.1 后端测试
16.2 前端测试（Flutter + Admin）
16.3 性能测试
16.4 CI 集成要点
附录 A：错误码测试映射
附录 B：测试命名约定

一、测试目标与原则

1.1 目标

1. 确保六条核心链路（资产、合同、账单、收款、工单、二房东填报）端到端闭环无阻塞
2. 保证财务计算精度（NOI、WALE、租金递增、KPI 打分）与手工 Excel 对照误差 < 0.01
3. 验证全部 4 套状态机（合同 12 条转移 / 工单 11 条 / 账单 11 条 / 押金 7 条）的合法与非法转换路径
4. 验证 RBAC 权限矩阵 6 角色 × 200+ 端点的授权正确性
5. 验证二房东行级数据隔离不可绕过
6. 验证 PIPL 合规要求（加密存储、脱敏默认、审计可溯）

1.2 原则

原则	说明
测试金字塔	单元测试 > 集成测试 > E2E 测试，优先保证底层覆盖
纯函数优先	`rent_escalation_engine` / `kpi_scorer` 两个 package 100% 单元测试覆盖
隔离职责	Repository 测试用真实数据库（testcontainers）、Service 测试用 Mock Repository、Controller 测试用 Mock Service
可重复执行	每次测试使用全新数据库 schema，测试数据不依赖运行顺序
CI 绑定	所有测试在 PR 合并前必须通过，不允许跳过

二、测试分层策略

```
┌──────────────────────────────────────────────────┐
│  Layer 4: UAT 验收测试（手动 + 半自动化脚本）      │  ← PRD 验收标准
├──────────────────────────────────────────────────┤
│  Layer 3: API E2E 测试（HTTP 请求 → 数据库 → 响应）  │  ← API_CONTRACT 端点
```

```
├──────────────────────────────────────────────────────────┤
│  Layer 2: 集成测试（Service + Repository + DB）      │  ← 状态机、事务原子性
├──────────────────────────────────────────────────────────┤
│  Layer 1: 单元测试（纯函数 / Mock 依赖）          │  ← 计算引擎、Store
└──────────────────────────────────────────────────────────┘
```

层级	后端技术栈	前端技术栈（Flutter + Admin）
Layer 1 单元	`package:test` + `mocktail`	`vitest` + `@vue/test-utils` + `pinia` testing
Layer 2 集成	`package:test` + 真实 PostgreSQL（Docker）	`vitest` + MSW (Mock Service Worker)
Layer 3 E2E	`package:test` + `package:http` → 启动 Shelf Server	—
Layer 4 UAT	人工操作 + 验收脚本（seed 数据 + curl 断言）	人工操作

三、覆盖率目标

范围	覆盖率目标	说明
`packages/rent_escalation_engine`	**≥ 95%**	6 种递增类型 + 混合分段 + 边界日期
`packages/kpi_scorer`	**≥ 95%**	正向/反向指标 + 满分/及格/零分/插值边界
`backend/lib/modules/*/services/`	**≥ 85%**	业务逻辑核心层
`backend/lib/modules/*/repositories/`	**≥ 80%**	SQL 正确性（需真实 DB）
`backend/lib/core/`	**≥ 90%**	中间件、错误处理、分页、加密
`app/src/stores/` + `admin/src/stores/`	**≥ 80%**	Pinia Store 状态与 action
`app/src/api/` + `admin/src/api/`	**≥ 75%**	API client 模块（Mock HTTP）

> 度量工具: 后端使用 `dart test --coverage` + `coverage` 包生成 lcov；前端使用 `vitest --coverage` 。CI 流水线中设置覆盖率卡点，低于目标则阻断合并。

四、单元测试计划

4.1 租金递增引擎 (`packages/rent_escalation_engine`)

用例编号	测试场景	输入	预期输出
RE-01	固定比例递增（年涨 5%）	基准 ¥80/m²，第 2 年	¥84.00/m²
RE-02	固定金额递增（年涨 ¥3/m²）	基准 ¥80/m²，第 3 年	¥86.00/m²
RE-03	阶梯式递增	第1~2年 ¥80，第3~4年 ¥90，第5年 ¥100	各阶段精确匹配
RE-04	CPI 挂钩递增	基准 ¥100，CPI 2.3%	¥102.30
RE-05	每 N 年递增（每 2 年涨 8%）	基准 ¥100，第 3 年	¥108.00
RE-06	免租后基准调整	免租 3 月后 ¥75/m² 起步	免租期 ¥0，后 ¥75
RE-07	混合分段（3 阶段组合）	见 PRD 2.5 示例	各阶段串联正确
RE-08	跨月边界日期	2026-01-31 起租，月度周期	2/28 或闰年 2/29 正确处理
RE-09	首期不足月折算	1月15日起租	当期 = 月租 ÷ 31 × 17
RE-10	末期不足月折算	最后一期仅 10 天	当期 = 月租 ÷ 当月天数 × 10
RE-11	零基准租金（异常）	基准 ¥0	抛出参数校验异常
RE-12	负递增率（合法）	-5% 递增	正确递减

4.2 KPI 打分引擎 (`packages/kpi_scorer`)

用例编号	测试场景	输入	预期输出
KS-01	正向指标满分	出租率 actual=95%，满分阈值=95%	score=100
KS-02	正向指标及格	出租率 actual=80%，及格阈值=80%	score=60
KS-03	正向指标线性插值	actual=87.5%，满分95%，及格80%	score=80
KS-04	正向指标低于及格	actual=70%，及格80%	score < 60（线性递减至 0）
KS-05	反向指标满分	逾期率 actual=3%，满分阈值≤5%	score=100
KS-06	反向指标及格	逾期率 actual=10%，及格≤10%	score=60
KS-07	反向指标翻转插值	actual=7.5%，满分5%，及格10%	score=80
KS-08	反向指标超及格	actual=15%，及格10%	score < 60（递减至 0）
KS-09	边界值：actual = 满分阈值	精确等于	score=100
KS-10	边界值：actual = 及格阈值	精确等于	score=60
KS-11	权重加权计算	3 指标权重 0.4/0.3/0.3	总分 = Σ(score$i$ × $weight$i)
KS-12	权重总和 ≠ 1.0	权重 0.3/0.3/0.3	抛出 `WEIGHT_SUM_NOT_ONE`

4.3 WALE 计算 (`wale_service.dart`)

用例编号	测试场景	预期
WA-01	单合同收入加权 WALE	剩余天数 / 365
WA-02	单合同面积加权 WALE	剩余天数 / 365
WA-03	多合同加权平均	Σ(剩余租期i × 年化租金i) / Σ(年化租金_i)，精确到 0.01 年
WA-04	多单元合同（M:N）拆分计算	每个单元独立计入，不重复加权
WA-05	已终止合同剩余租期归零	terminated 合同不参与 WALE 计算
WA-06	已到期合同（expired）	剩余租期 = 0
WA-07	按楼栋级聚合	仅含该楼栋单元的合同
WA-08	按业态级聚合	仅含该业态单元的合同
WA-09	空合同集（全部空置）	WALE = 0
WA-10	剩余租期精确到天	与 Excel 手工计算对照误差 < 0.01 年

4.4 NOI 计算 (`noi_service.dart`)

用例编号	测试场景	预期
NO-01	基础 NOI = EGI - OpEx	精确匹配
NO-02	EGI = PGI - 空置损失 + 其他收入	精确匹配
NO-03	不含税口径	含税金额 ÷ (1 + tax_rate)
NO-04	押金不计入 NOI	押金收取/退还不影响 NOI 数值
NO-05	按楼栋下钻	仅含该楼栋资产的收支
NO-06	按业态下钻	三业态独立计算
NO-07	免租期账单标记 exempt	不计入 PGI
NO-08	已作废账单	cancelled 不计入
NO-09	空置损失 = 空置单元 × 市场参考租金	按 `market_rent_reference` 计算

4.5 信用评级 (`credit_rating_service.dart`)

用例编号	测试场景	预期评级
CR-01	12 个月 0 次逾期	A
CR-02	12 个月 1 次逾期，3 天	A
CR-03	12 个月 2 次逾期，单次 ≤ 15 天	B
CR-04	12 个月 3 次逾期	B
CR-05	12 个月 4 次逾期	C
CR-06	12 个月 1 次逾期 > 15 天	C
CR-07	新租户（签约 < 3 月）	B（默认，不评级）
CR-08	签约恰好 3 个月	触发首次评级

4.6 核心 Service 层测试

以下 Service 使用 `mocktail` Mock Repository 接口进行测试：

Service	关键测试场景
`ContractService`	创建合同（单/多单元）、状态机每条合法转换、拒绝非法转换、提前终止 4 种类型触发副作用
`DepositService`	收取 → 冻结 → 冲抵 → 退还全路径、扣除超额异常、续签滚转
`InvoiceService`	自动生成（含免租期 exempt）、逾期标记、作废、部分收款核销

`ReceivableService`	一笔收款核销多账单、部分收款、分配总额校验
`MeterReadingService`	抄表录入校验（本期 > 上期）、阶梯水电计费、自动生成 utility 账单
`TurnoverService`	营业额申报 → 审核 → 分成账单生成；补报重算差额
`WorkOrderService`	全状态机路径（含挂起恢复）、验收不通过返工、完工成本归口
`SubleaseService`	审核流（draft → pending → approved/rejected）、面积超限校验、版本留痕
`AlertService`	8 种预警类型触发正确、同日防重复轰炸、失败重试
`OrganizationService`	3 级组织树 CRUD、层级超限校验、停用前检查活跃员工/子部门
`KpiService`	方案绑定、自动取数+打分、快照冻结、申诉流程、排名计算

五、集成测试计划

集成测试使用 Docker 启动真实 PostgreSQL 实例，执行完整迁移脚本后运行。

5.1 数据库迁移验证

用例编号	测试场景	预期
MG-01	顺序执行 001~017 迁移脚本	无 SQL 错误
MG-02	全部枚举类型创建成功	32 种 ENUM 可查询
MG-03	外键约束完整性	40+ 表的 FK 引用目标表存在
MG-04	索引创建成功	8 个性能关键索引可用
MG-05	种子数据插入	`seed.sql` 执行无冲突
MG-06	回滚脚本（反向执行）	逐步 DROP 无残留

5.2 Repository 集成测试

模块	关键测试场景
`UserRepository`	CRUD、邮箱唯一约束、角色过滤、锁定状态更新
`ContractRepository`	创建含 contract*units M:N* 关联、续签链查询（*parent*contract_id）、按状态+楼栋筛选
`InvoiceRepository`	按合同+账期查询、逾期账单批量更新、按状态统计
`DepositRepository`	押金状态流转持久化、deposit_transactions 流水写入
`SubleaseRepository`	行级隔离验证：二房东 A 调用不返回二房东 B 数据
`UnitRepository`	按楼层+状态查询（热区渲染）、归档单元不出现在可租列表
`KpiRepository`	快照写入与读取、历史趋势查询（6~12 月）、排名聚合
`AuditLogRepository`	按 resource*type* + *resource*id 查询、时间范围过滤

5.3 事务原子性测试

用例编号	场景	预期
TX-01	合同签约（T3）：单元状态更新 + contract_units 写入 + 账单生成	全部成功或全部回滚
TX-02	合同终止（T6）：单元释放 + 账单取消 + 押金冻结	全部成功或全部回滚
TX-03	收款核销：payment + 多条 payment_allocation	分配总额 ≠ 收款额时全部回滚
TX-04	押金续签转移（D3）：原合同 refunded + 新合同 collected	原子性保证
TX-05	Excel 批量导入（整批回滚模式）	一条失败全部回滚

六、API 端到端测试计划

启动完整 Shelf HTTP Server，通过 HTTP Client 发送真实请求，验证请求 → 中间件 → Controller → Service → Repository → DB → 响应的完整链路。

6.1 认证链路

用例编号	场景	预期
AE-01	正常登录	200 + access*token* + *refresh*token
AE-02	错误密码	401 + `INVALID_CREDENTIALS`
AE-03	连续 5 次错误密码	423 + `ACCOUNT_LOCKED` (locked_until)
AE-04	锁定期内登录	423 + `ACCOUNT_LOCKED`
AE-05	Token 刷新	200 + 新 token
AE-06	过期 Token 访问	401 + `TOKEN_EXPIRED`
AE-07	改密后旧 Token 失效	401 + `SESSION_VERSION_MISMATCH`
AE-08	二房东账号主合同到期后	403 + `ACCOUNT_FROZEN`
AE-09	密码复杂度不足	400 + `PASSWORD_TOO_WEAK`

6.2 RBAC 权限验证

对 6 个角色 × 核心端点组合进行矩阵测试:

角色	可访问端点（样本）	禁止端点（样本）
`super_admin`	全部 200+ 端点	无
`ops_manager`	资产/合同/财务 R+W、工单审批、KPI 管理	用户管理（403）
`leasing_specialist`	合同/租客 R+W、财务只读	账单核销（403）、用户管理（403）
`finance_staff`	财务 R+W、核销、抄表	合同写（403）、用户管理（403）
`frontline_staff` → `maintenance_staff`	工单创建+完工、水电抄表	合同写（403）、财务写（403）
`property_inspector`	资产只读、工单只读、水电抄表	合同写（403）、财务写（403）、工单写（403）
`report_viewer`	资产/合同/财务只读、NOI/WALE	任何写操作（403）
`sub_landlord`	`/sublease-portal/*`	其余所有端点（403）

> 实现方式: 使用参数化测试（parameterized test），遍历 角色 × 端点 × 预期状态码 三元组。

6.3 响应信封验证

用例编号	场景	预期
ENV-01	列表接口成功	`{ "data": [...], "meta": { "page": 1, "pageSize": 20, "total": N } }`
ENV-02	详情接口成功	`{ "data": { ... } }`
ENV-03	业务错误	`{ "error": { "code": "SCREAMING_SNAKE", "message": "..." } }`
ENV-04	分页超限 (pageSize > 100)	400 + `VALIDATION_ERROR`
ENV-05	无效 JSON body	400 + `VALIDATION_ERROR`

6.4 核心业务链路 E2E

用例编号	链路	步骤
BL-01	合同全生命周期	创建租客 → 创建合同(quoting) → confirm*quote(pending*sign) → sign(active) → 定时任务(expiring_soon) → renew(renewed)
BL-02	合同提前终止	active → terminate(tenant*early*exit) → 验证：单元=vacant、未来账单=cancelled、押金=frozen、WALE 归零
BL-03	账单收款闭环	合同签约 → 自动生成 draft 账单 → issue → 创建收款 → 核销分配 → paid
BL-04	部分收款与跨账单核销	2 张 issued 账单 → 1 笔大额收款 → 分配核销 2 张
BL-05	工单完整流转	提交 → 审批派单 → 开始处理 → 提交完工 → 通过验收 → 费用归口
BL-06	工单挂起与恢复	in*progress* → *hold* → *resume* → *in*progress
BL-07	二房东填报+审核	二房东登录 → 填报 sublease(draft) → 提交(pending) → 运营审核(approved) → 进入看板
BL-08	押金全流程	收取(collected) → 合同终止触发冻结(frozen) → 扣除违约金(partially_credited) → 退还余额(refunded)
BL-09	水电抄表→账单	录入抄表读数 → 计费（含阶梯价） → 自动生成 utility 账单
BL-10	营业额分成→账单	商户申报 → 财务审核 → 生成分成账单（取 MAX(保底, 分成额)）
BL-11	KPI 评分全流程	配置方案 → 绑定对象 → 自动取数打分 → 冻结快照 → 员工申诉 → 审核重算
BL-12	Excel 批量导入	上传单元 Excel → 试导入(dry_run) → 正式导入 → 重复导入被拒
BL-13	押金续签滚转	原合同续签 → 押金 transfer → 原合同 refunded + 新合同 collected
BL-14	通知全流程（v1.8 新增）	业务触发 → 写入 notifications → 查询列表（分页/筛选）→ 标记已读 → 全部已读 → 未读数归零
BL-15	审批全流程（v1.8 新增）	创建审批请求 → 查询待审批列表 → 通过审批 → 验证通知已发 + 业务状态已变更
BL-16	审批拒绝（v1.8 新增）	创建审批请求 → 拒绝（含 remark）→ 验证 rejection 通知已发
BL-17	审批自审拒绝（v1.8 新增）	用户自己审批自己提交的申请 → 403 `APPROVAL_SELF_REVIEW`
BL-18	催收记录管理（v1.8 新增）	创建催收记录（关联逾期账单）→ 查询列表（筛选 method/tenant）→ 验证通知已发
BL-19	Dashboard 聚合（v1.8 新增）	GET /api/dashboard/overview → 验证返回出租率/NOI/WALE/未读通知数等聚合数据

七、状态机转换测试矩阵

7.1 合同状态机（7 态 12 条转移）

对每条转移路径测试：合法转换成功、前置条件不满足时拒绝、副作用正确执行。

转移	合法测试	非法/边界测试
T1 quoting → pending_sign	至少 1 个可租单元 + 月租金 > 0	无单元关联 → 拒绝；月租金=0 → 拒绝
T2 quoting → 删除	无签约动作	已有签约动作 → 拒绝
T3 pending_sign → active	已上传 PDF + 押金已收 + 单元无冲突	押金未收 → 拒绝；单元已被占 → `UNIT_ALREADY_LEASED`
T4 pending_sign → quoting	签约退回	—
T5 active → expiring_soon	剩余 ≤ 90 天（定时任务）	剩余 91 天 → 不转换
T6 active → terminated	4 种 termination_type	无权限 → 403；合同非 active → `CONTRACT_NOT_ACTIVE`
T7 expiring_soon → renewed	已创建续签合同	无续签合同 → 拒绝
T8 expiring_soon → expired	end_date < NOW()	end_date ≥ NOW() → 不转换
T9 expiring_soon → terminated	同 T6	同 T6
T10 expired → *	—	任何转换 → `INVALID_STATUS_TRANSITION`
T11 renewed → *	—	任何转换 → `INVALID_STATUS_TRANSITION`
T12 terminated → *	—	任何转换 → `INVALID_STATUS_TRANSITION`

7.2 工单状态机（7 态 11 条转移）

转移	合法测试	非法测试
W1 submitted → approved	有审核权限 + 指派处理人	无权限 → 403
W2 submitted → rejected	有审核权限 + 拒绝原因	无原因 → 400
W3 approved → in_progress	处理人确认	—
W4 approved → on_hold	挂起原因	无原因 → 400
W5 in*progress* → *pending*inspection	提交完工 + 可选照片	—
W6 in*progress* → *on*hold	挂起原因	—
W7 pending_inspection → completed	验收通过 + 费用记录	—
W8 pending*inspection* → *in*progress	验收不通过返工	—
W9 on*hold* → *approved*/*in*progress	恢复到挂起前	—
W10 rejected → *	终态	任何转换 → 拒绝
W11 completed → *	终态	任何转换 → 拒绝

7.3 账单状态机（6 态 11 条转移）

转移	合法测试	非法测试
I1 draft → issued	金额 > 0 + 账期已到	金额=0 → 拒绝
I2 draft → exempt	免租期内	非免租期 → 拒绝
I3 draft → cancelled	合同终止触发	—
I4 issued → paid	outstanding_amount = 0	仍有余额 → 保持 issued
I5 issued → overdue	due_date < NOW()（定时任务）	due_date ≥ NOW() → 不转换
I6 issued → cancelled	误开作废	—
I7 overdue → paid	全额核销	—
I8 overdue → cancelled	特殊作废	—
I9~I11 终态	—	paid/cancelled/exempt → 任何转换拒绝

7.4 押金状态机（4 态 7 条转移）

转移	合法测试	非法测试
D1 collected → collected (扣除)	金额 ≤ 余额	超额 → `DEDUCTION_EXCEEDS_BALANCE`
D2 collected → frozen	合同终止触发	—
D3 collected → transfer	续签合同已创建	非续签 → `TARGET_CONTRACT_NOT_RENEWAL`
D4 frozen → partially_credited	扣除后余额 > 0	—
D5 frozen → refunded	无未结账单	有未结 → `CONTRACT_HAS_OUTSTANDING_INVOICES`
D6 partially_credited → refunded	无未结账单	同上
D7 refunded → *	终态	任何转换 → 拒绝

八、业务计算精度测试

使用手工 **Excel** 计算对照验证系统精度，以下为验收样本集设计：

8.1 WALE 验收样本

样本集	合同数	包含场景	精度要求
WALE-S1	5	单单元、单业态	收入 WALE 误差 < 0.01 年
WALE-S2	10	多单元合同（M:N)、三业态混合	面积 WALE 误差 < 0.01 年
WALE-S3	20	含到期、续签、终止合同	终止合同剩余租期=0，不参与计算

8.2 NOI 验收样本

样本集	场景	验证点
NOI-S1	单楼栋月度 NOI	EGI - OpEx = NOI
NOI-S2	三业态分拆	各业态收入+支出独立归账
NOI-S3	含免租期+空置损失	exempt 不计 PGI；空置按 market_rent 估算
NOI-S4	含税/不含税转换	NOI 使用不含税口径

8.3 递增规则验收样本

样本集	递增类型	合同期限	验证点
ESC-S1	固定比例 5%/年	3 年	每年实际金额与手工计算一致
ESC-S2	阶梯式 3 段	5 年	各段切换时间点精确
ESC-S3	混合递增（免租+固定+CPI）	5 年	3 阶段串联，CPI 手工录入后匹配

8.4 KPI 打分验收样本

样本集	方案	验证点
KPI-S1	3 正向指标	总分 = Σ(score$i$ × $weight$i) 与手工一致
KPI-S2	含反向指标	反向指标线性插值翻转正确
KPI-S3	跨期快照	上月快照不受本月方案修改影响

九、安全测试计划

9.1 OWASP Top 10 清单

#	风险类别	测试场景	预期
A01	访问控制失效	二房东 A 通过修改 URL 参数访问二房东 B 数据	403 或空结果
A01	访问控制失效	maintenance_staff 角色访问 `/api/users`	403
A01	访问控制失效	修改 JWT payload 中 role 字段	签名验证失败 401
A02	加密失效	直接查询数据库 `cert_no_encrypted` 字段	密文不可读，非明文
A02	加密失效	API 默认返回 `cert_no`	仅后 4 位可见
A02	加密失效	脱敏还原需二次验证	验证失败 → `INVALID_PASSWORD`
A02	加密失效	脱敏还原审计	`audit_logs` 记录查看操作
A03	注入	SQL 注入：`' OR 1=1 --` 参数化查询拦截	参数不被拼接入 SQL
A03	注入	XSS：`<script>alert(1)</script>` 写入备注字段	输出时转义，不执行
A04	不安全设计	批量导入恶意文件（非 Excel）	文件类型校验拒绝
A05	安全配置错误	生产环境暴露 stack trace	500 响应只含 `INTERNAL_ERROR`，不含堆栈
A05	安全配置错误	缺少 CORS 限制	响应头正确设置 `Access-Control-Allow-Origin`
A07	身份验证失败	JWT 过期后仍可使用	401
A07	身份验证失败	暴力破解（1000 次/分钟）	速率限制 60 req/min/IP
A08	软件数据完整性	文件上传：伪装 `.jpg` 扩展名的 `.exe`	魔术字节 MIME 校验拒绝
A09	日志与监控不足	合同变更、权限变更是否记审计日志	`audit_logs` 表有对应记录

9.2 二房东数据隔离专项

用例编号	场景	预期
ISO-01	二房东 A 查询 sublease 列表	仅返回 master*contract*id 匹配的记录
ISO-02	二房东 A 通过 ID 直接访问 B 的 sublease	404（不是 403，防止信息泄露）
ISO-03	二房东 A 通过 unit_id 访问非授权单元	404
ISO-04	二房东 A 修改请求 body 中的 master*contract*id	被 Repository 层覆盖为 JWT 中绑定值
ISO-05	主合同到期 → 二房东账号冻结	冻结后登录返回 `ACCOUNT_FROZEN`

9.3 文件安全

用例编号	场景	预期
FS-01	路径穿越 `../../etc/passwd`	路径规范化后拒绝
FS-02	超大文件上传（> MAX*UPLOAD*SIZE_MB）	413 + `FILE_TOO_LARGE`
FS-03	非法文件类型（.exe / .sh）	415 + `FILE_TYPE_NOT_ALLOWED`
FS-04	无权访问他人合同 PDF	403 + `FILE_ACCESS_DENIED`

9.4 PIPL 合规验证

用例编号	场景	预期
PP-01	租户证件号 API 返回格式	`****1234`（脱敏后 4 位）
PP-02	手机号 API 返回格式	`****5678`（脱敏后 4 位）
PP-03	脱敏还原操作写审计日志	audit*logs* 记录 *user*id + action= `tenant.unmask` + timestamp
PP-04	合同终止 3 年后租户个人信息	`data_retention_until` 字段正确设置
PP-05	二房东填报终端租客信息	提交时系统提示数据处理授权须知

十、性能测试计划

基于 PRD 非功能性要求定义性能基准（Phase 1 规模：639 房源、~500 合同、50 并发用户）。

10.1 API 响应时间 SLA

场景	P95 响应时间	P99 响应时间
仪表盘加载（NOI + 出租率 + WALE）	< 2 秒	< 3 秒
单元列表（639 条，含分页）	< 500ms	< 1 秒
合同列表（500 条，含分页）	< 500ms	< 1 秒
WALE 计算（全量 500 合同）	< 1 秒	< 2 秒
楼层热区图渲染（SVG + 状态色块）	< 1 秒	< 2 秒
账单详情（含 invoice_items）	< 300ms	< 500ms
二房东门户首页加载	< 1 秒	< 2 秒

10.2 批量操作性能

场景	目标
账单批量生成（639 条/批次）	< 30 秒
Excel 导入 639 套单元	< 60 秒
KPI 全员评分（~50 人）	< 10 秒
信用评级全量重算（~300 租户）	< 30 秒

10.3 并发测试

场景	并发数	目标
混合读写（70% 读 30% 写）	50	无 5xx 错误，P95 < 2 秒
登录接口	20	P95 < 1 秒
仪表盘并发刷新	30	P95 < 3 秒

10.4 数据库查询优化验证

查询场景	验证方式	目标
楼层色块渲染	EXPLAIN ANALYZE	使用 `units(floor_id, current_status)` 索引
WALE 计算	EXPLAIN ANALYZE	使用 `contracts(status, end_date)` 索引
逾期账单查询	EXPLAIN ANALYZE	使用 `invoices(status, due_date)` 部分索引
二房东数据查询	EXPLAIN ANALYZE	使用 `subleases(master_contract_id)` 索引
审计日志查询	EXPLAIN ANALYZE	使用 `audit_logs(resource_type, resource_id)` 索引

十一、前端测试计划（Flutter + Admin）

11.1 BLoC/Cubit 单元测试（Flutter）+ Pinia Store 单元测试（Admin）

Flutter 端使用 `bloc_test` + `mocktail`，验证 BLoC/Cubit 状态转换。Admin 端使用 `vitest` + `pinia` testing helpers，验证 Store action 执行后 state 变化。Mock API client（不依赖 HTTP）。

Store	关键测试场景
`useAuthStore`	登录成功(→token 写入)、登录失败(→error)、token 刷新、登出

`useContractListStore`	加载(loading→数据填充)、筛选、分页、搜索
`useContractFormStore`	表单验证、提交成功、提交失败、多单元添加/移除
`useInvoiceStore`	加载、状态筛选、核销后刷新
`useWorkOrderStore`	创建、状态流转、照片上传
`useSubleaseStore`	填报、提交审核、审核通过/退回
`useKpiStore`	方案加载、排名展示、快照切换、趋势数据
`useDepositStore`	押金列表、冻结/扣除/退还操作
`useFloorMapStore`	SVG 加载、热区状态色块映射、单元点击

11.2 组件测试

Flutter 端使用 `flutter_test` + `mocktail`，通过 `BlocProvider` 注入 mock BLoC 验证 UI 渲染。Admin 端使用 `@vue/test-utils` + `vitest`，通过 `createTestingPinia` 注入初始 state，验证 UI 渲染。

组件 / 页面	测试重点
`LoginPage`	输入校验、密码可见性切换、错误提示
`ContractDetail`	各状态操作按钮可见性（active 显示终止按钮、terminated 无操作按钮）
`FloorMap`	SVG 渲染 + 状态色块颜色映射（leased→绿、vacant→红、expiring→黄）
`InvoiceDetail`	金额格式、含税/不含税双金额展示
`KpiDashboard`	loading / loaded / error 三态渲染正确
`PaymentForm`	多账单核销分配表单、金额校验

11.3 状态色块映射验证

验证色彩语义与业务状态正确映射（遵循 copilot-instructions 色彩规范）：

状态	Flutter ThemeExtension	Admin Element Plus	测试验证
`leased` / `paid`	`colorScheme.primary` / extension `success`（绿色系）	`type="success"`	已租单元/已核销账单为绿
`expiring_soon` / `warning`	`colorScheme.tertiary` / extension `warning`（黄/橙色系）	`type="warning"`	即将到期为黄
`vacant` / `overdue` / `error`	`colorScheme.error`（红色系）	`type="danger"`	空置/逾期为红
`non_leasable`	`colorScheme.outline`（灰色）	`type="info"`	非可租为灰

十二、测试数据策略

12.1 种子数据覆盖范围

基于 `scripts/seed.sql` 构建完整测试数据集：

数据类别	数量	覆盖场景
楼栋	3	A 座写字楼、商铺区、公寓楼
楼层	15	A 座 10 层 + 商铺 2 层 + 公寓 3 层
单元	50（测试子集）	三业态各覆盖：leased/vacant/expiring*soon*/*non*leasable
租客	20	含企业/个人、信用 A/B/C/D 各级
合同	30	含全部 7 种状态、单/多单元、4 种终止类型
账单	100	含 6 种状态（draft/issued/paid/overdue/cancelled/exempt）
收款记录	30	含全额/部分/跨账单核销
押金	10	含 4 种状态
工单	20	含全部 7 种状态
子租赁	15	含 4 种审核状态
用户	10	覆盖 6 种角色
部门	5	3 级组织树
KPI 方案	2	绑定不同对象的方案

12.2 边界值测试数据

场景	测试数据
合同起租日 = 月末（2/28, 2/29, 1/31）	验证账期跨月处理
合同期限 = 1 天	验证极端短租
合同期限 = 10 年	验证长租递增计算
账单金额 = 0.01	验证最小金额
账单金额 = 9,999,999.99	验证大金额
单元面积 = 0	验证参数校验拒绝
639 套单元全量导入	验证批量性能
同一合同绑定 10 个单元	验证 M:N 极端场景
递增规则 10 个阶段	验证混合递增阶段数

12.3 Mock 数据生成规则

字段类型	生成规则
UUID	`gen_random_uuid()` 自动生成
证件号	测试用固定格式 `110101200001010001`，加密后存储
手机号	测试用 `13800138000` ~ `13800138099`
金额	使用 Decimal 精度（2 位小数），避免浮点误差
日期	使用固定 `Clock` 注入，不依赖 `DateTime.now()`
枚举	覆盖每个枚举值至少 1 条数据

十三、UAT 验收标准

基于 PRD 第七节验收标准，定义具体执行步骤：

13.1 资产模块

验收编号	步骤	通过标准
UAT-A01	上传验收用 CAD 文件（.dwg）→ 查看 SVG 转换结果	SVG 展示完整，无缺失图层
UAT-A02	随机抽取 30 个单元，点击热区	热区与 unit_id 一一对应，正确率 100%
UAT-A03	修改合同状态（active → terminated），刷新楼层图	1 分钟内对应单元色块由绿变红
UAT-A04	执行 639 套单元 Excel 导入	无错误，数据一致
UAT-A05	导入含错误数据的 Excel	返回错误明细（行号+字段+原因），无脏数据入库
UAT-A06	试导入模式（dry_run）	仅校验不入库，报告校验结果

13.2 租务与合同

验收编号	步骤	通过标准
UAT-C01	创建多单元合同（3 个单元）	contract_units 正确关联，各单元独立价格
UAT-C02	3 类递增样本手工验算	系统金额与 Excel 计算一致
UAT-C03	WALE 双口径计算（20 份样本合同）	收入 WALE 与面积 WALE 误差均 < 0.01 年
UAT-C04	合同提前终止（租户退租）	单元→vacant、账单→cancelled、押金→frozen、WALE 归零
UAT-C05	合同续签 + 押金滚转	原合同 refunded、新合同 collected，金额一致
UAT-C06	到期 ≤30 天合同	10 分钟内触发预警通知
UAT-C07	逾期账单（超 due_date 1/7/15 天）	分别触发 3 级预警

13.3 财务与 NOI

验收编号	步骤	通过标准
UAT-F01	自动生成月度账单	含租金+物管费+水电费项
UAT-F02	一笔收款核销 2 张账单	分配正确，account balance 归零
UAT-F03	部分收款	outstanding_amount 正确更新
UAT-F04	NOI 看板 = 手工核算	EGI - OpEx = NOI，三业态分拆正确
UAT-F05	含税/不含税双金额	账单同时展示两种金额
UAT-F06	水电抄表 → 自动生成账单	含阶梯价正确计算
UAT-F07	营业额分成	MAX(保底, 分成额) 正确取高
UAT-F08	押金不计入 NOI	押金变动不影响 NOI 数值

13.4 工单

验收编号	步骤	通过标准
UAT-W01	移动端提交报修（含照片）	成功创建工单
UAT-W02	PC 端派单+处理+验收	全流程完整流转
UAT-W03	验收不通过 → 返工 → 再验收	状态正确回退与前进
UAT-W04	完工成本归口	自动生成 expense 记录

13.5 二房东穿透

验收编号	步骤	通过标准
UAT-S01	二房东登录 → 填报 → 提交 → 运营审核通过	全流程闭环
UAT-S02	审核退回 → 重新提交	版本留痕正确
UAT-S03	二房东 A 无法查看 B 的数据	行级隔离验证通过
UAT-S04	主合同到期 → 二房东自动冻结	冻结后无法登录
UAT-S05	连续 5 次错误密码	锁定 30 分钟
UAT-S06	抽样 20 条子租赁记录	与手工核对一致

13.6 KPI 考核

验收编号	步骤	通过标准
UAT-K01	配置 2 套 KPI 方案，绑定不同部门	方案独立，权重和=1
UAT-K02	自动取数打分	与手工 Excel 计算一致
UAT-K03	冻结评分快照	历史快照可回看，不受方案修改影响
UAT-K04	员工申诉 → 管理层审核 → 批准重算	全链路审计日志
UAT-K05	排名展示	排名榜正确排序
UAT-K06	Excel 导出评分明细	含各指标实际值/得分/加权得分

13.7 安全验收

验收编号	步骤	通过标准
UAT-SEC01	查看租户详情 API 响应	证件号/手机号 `****xxxx` 脱敏
UAT-SEC02	请求脱敏还原	需二次密码验证 + 写审计日志
UAT-SEC03	直接查询 DB `cert_no_encrypted`	密文不可读
UAT-SEC04	二房东外部门户	仅 HTTPS 可访问
UAT-SEC05	审计日志覆盖	合同变更、账单核销、权限变更、押金操作、KPI 申诉全部有日志

十四、定时任务测试

14.1 任务执行正确性

任务	测试方法	验证点
J1 billing_generation	创建 active 合同 → 手动触发 → 检查账单	draft 账单生成正确，含免租期 exempt
J2 contract*expiry*check	创建 end_date = 明天的合同 → 触发	active → expiring*soon*；*end*date < 今天 → expired
J3 invoice*overdue*check	创建 due_date = 昨天的 issued 账单 → 触发	issued → overdue
J4 alert_engine	创建符合预警条件的合同/账单 → 触发	alerts 表有记录 + 通知发送
J5 deposit*refund*reminder	创建 7 天后终止的合同 → 触发	提醒记录生成
J6 credit*rating*recalc	创建有逾期记录的租户 → 触发	评级正确（A/B/C/D）

14.2 幂等性与防重

用例编号	场景	预期
JOB-01	同一任务重复执行 2 次	不生成重复账单/预警
JOB-02	同日同合同同预警类型	仅触发 1 次（防轰炸）

14.3 失败与重试

用例编号	场景	预期
JOB-03	任务执行中断（模拟异常）	job*execution*logs 记录 failed
JOB-04	自动重试（3 次指数退避）	60s → 120s → 180s 间隔重试
JOB-05	重试耗尽	标记 retry_exhausted，等待人工补偿
JOB-06	人工触发补偿	POST `/api/jobs/:jobName/trigger` 成功执行

14.4 执行顺序依赖

用例编号	场景	预期
JOB-07	billing*generation(00:30)* 先于 *invoice*overdue_check(01:00)	先生成账单再检查逾期
JOB-08	contract*expiry*check 先于 alert_engine	先更新合同状态再触发预警

十五、回归测试清单

每次代码合并前必须通过的核心回归用例（约 30 条，CI 自动执行）：

#	回归用例	涉及模块	优先级
R01	登录 + Token 刷新	Auth	P0
R02	RBAC 6 角色 × 核心端点矩阵	Core	P0
R03	合同 CRUD + 状态机（T1~T6）	Contracts	P0
R04	合同终止副作用（单元/账单/押金/WALE）	Contracts+Finance	P0
R05	账单自动生成	Finance	P0
R06	收款核销（全额+部分+跨账单）	Finance	P0
R07	WALE 双口径计算	Contracts	P0
R08	NOI 计算	Finance	P0
R09	递增引擎 6 类型	Package	P0
R10	KPI 打分（正向+反向）	Package	P0
R11	工单完整流转	Workorders	P0
R12	二房东数据隔离	Subleases	P0
R13	押金全流程	Contracts	P1
R14	水电抄表→账单	Finance	P1
R15	营业额分成	Finance	P1
R16	KPI 申诉流程	Finance/KPI	P1
R17	Excel 导入（试导入+正式+回滚）	Import	P1
R18	CAD 上传转换	Assets	P1
R19	信用评级计算	Contracts	P1
R20	预警触发+防重	Jobs	P1
R21	脱敏默认+还原审计	Security	P1
R22	密码复杂度+锁定+冻结	Auth	P1
R23	组织架构 3 级 CRUD	Organization	P2
R24	管辖范围配置	Organization	P2
R25	分页参数校验	Core	P2
R26	文件上传安全（类型+大小+路径穿越）	Files	P2
R27	响应信封格式一致性	Core	P2
R28	审计日志覆盖完整性	Core	P2

R29	事务原子性（合同签约/终止）	DB	P2
R30	前端 Store 核心状态转换	Frontend	P2

十六、工具链与运行方式

16.1 后端测试

```
# 运行全部单元测试
cd backend && dart test test/unit/

# 运行集成测试（需 Docker 运行 PostgreSQL）
cd backend && dart test test/integration/

# 运行含覆盖率
cd backend && dart test --coverage=coverage/
dart pub global run coverage:format_coverage \
  --lcov --in=coverage --out=coverage/lcov.info --report-on=lib

# 运行 package 测试
cd backend/packages/rent_escalation_engine && dart test
cd backend/packages/kpi_scorer && dart test

# API E2E 测试
cd backend && dart test test/e2e/
```

16.2 前端测试（Flutter + Admin）

```
# Flutter BLoC/Cubit 单元测试
cd flutter_app && flutter test

# 含覆盖率
cd flutter_app && flutter test --coverage

# Admin Store + 组件单元测试
cd admin && pnpm test

# 含覆盖率
cd admin && pnpm test -- --coverage
```

16.3 性能测试

```
# 使用 wrk 或 k6 进行 HTTP 性能测试
k6 run test/performance/api_load_test.js

# 数据库查询分析
```

```
psql -d propos -c "EXPLAIN ANALYZE <query>"
```

16.4 CI 集成要点

阶段	执行内容	卡点
PR 检查	单元测试 + 覆盖率检查	覆盖率低于目标阻断合并
PR 检查	集成测试（Docker PostgreSQL）	任何失败阻断合并
合并后	E2E 测试 + 安全扫描	失败发告警
Release	性能测试 + UAT 脚本	手动审批

附录 A：错误码测试映射

完整错误码列表见 ERROR_CODE_REGISTRY.md。每个错误码至少有 1 个 E2E 测试用例覆盖其触发场景。

模块	错误码数量	测试覆盖要求
通用	6	100%（每个都有触发用例）
认证	7	100%
用户	6	100%
组织架构	5	100%
资产	6	100%
合同	10	100%
押金	4	100%
财务	8	100%
KPI	6	100%
工单	3	100%
二房东	4	100%
文件	6	100%
NOI 预算（v1.8 新增）	1	100%
催收（v1.8 新增）	1	100%
审批（v1.8 新增）	3	100%

附录 B：测试命名约定

```
// 后端测试文件命名
test/unit/<module>/<service>_test.dart
test/integration/<module>/<repository>_test.dart
test/e2e/<module>/<feature>_e2e_test.dart

// 前端测试文件命名（Flutter 与 Admin 同规范）
src/stores/__tests__/<store>.test.ts
src/components/__tests__/<component>.test.ts
src/pages/__tests__/<page>.test.ts       // Admin 为 src/views/__tests__/<view>.test.ts

// 测试方法命名：should_<预期行为>_when_<前置条件>
test('should return 403 when maintenance_staff accesses finance write', () { ... });
test('should calculate WALE correctly when multi-unit contracts exist', () { ... });
```

测试计划如有疑问或需细化某个模块的用例，请联系项目负责人。